

BACO / TennisLine

Local Setup Guide - running the site on a private laptop

This guide brings the full app up on a fresh machine: MySQL database, FastAPI backend, and the Next.js frontend. The GitHub repo has the **code**; you must also provide the **database** (data is not in git) and the **backend/.env** file (git-ignored). **.venv**, **node_modules** and **.next** are installed fresh.

1. Install prerequisites

- Git
- Python 3.12
- Node.js 20 LTS (Next.js 15 needs Node >= 18.18)
- MySQL 8+ (9.x is fine)

2. Get the code

```
git clone https://github.com/erezsh98/Baco-New.git
cd Baco-New
```

3. Database - bring the data over (recommended)

The app needs the **play_tennis** DB (clubs, users, schedules, tickets, roles). Easiest is a like-for-like dump from your current machine, which already has every schema change applied.

First, install MySQL 8 locally (Windows - no local MySQL yet):

- Download **MySQL Installer for Windows** (Community) from dev.mysql.com.
- Choose **Server only** (or Server + MySQL Shell). Install it as a Windows service (MySQL80) on port **3306**.
- Set a **root password** and remember it - it goes into backend/.env.

This is a separate LOCAL server; it does NOT affect a TOAD connection to the old GCP database (that is remote). Keep the new app pointed at localhost only - never run the import, create_tables.py, the migration script, or a rebuild against the GCP production DB. You can add a second TOAD connection to localhost:3306 to browse the new DB with the same tool.

On the current machine (export):

```
mysqldump -u root -p --single-transaction --routines --triggers \
--default-character-set=utf8mb4 play_tennis > play_tennis.sql
```

On the laptop (import):

```
mysql -u root -p -e "CREATE DATABASE play_tennis CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
mysql -u root -p --default-character-set=utf8mb4 play_tennis < play_tennis.sql
```

Alternative (empty start): skip the dump and run create_tables.py in step 4; you get empty tables and must create clubs/users/schedules by hand. Not ideal for testing the real site.

4. Backend

```
cd backend
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
```

Create **backend/.env** (not in the repo). Minimum for local testing:

```
DATABASE_URL=mysql+pymysql://root:YOUR_MYSQL_PASSWORD@localhost/play_tennis
JWT_SECRET=any-long-random-string
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=1440
DEV_MODE=true
```

- **DEV_MODE=true** bypasses the Pelecard gateway so credit-card bookings auto-confirm - test booking end to end.
- SMTP / Pelecard / SMS keys are optional; leave them out and those features simply no-op.

If you started with an **empty** DB (no dump imported):

```
python create_tables.py
python set_super_admin.py your@email.com
```

If you imported the dump, the tables, data and the super-admin role are already there - skip these.
Start it:

```
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

5. Frontend (new terminal)

```
cd frontend
npm install
npm run dev
```

The frontend proxies `/api/backend/*` to `http://localhost:8000` (overridable with `BACKEND_URL`), so keep the backend on port 8000.

6. Open the site

http://localhost:3000 - log in with an account that exists in your imported data.

7. Super-admin (one-time per environment)

Grant/assign the single super-user (manages clubs, managers, tickets/groups, availability rebuild):

```
cd backend
python set_super_admin.py your@email.com
```

Re-run with a different email to change the super-user. It creates the role if missing, revokes it from everyone else, and grants it to the given user. Data change only - no migration.

8. Testing the Pelecard payment gateway (optional)

By default **DEV_MODE=true** bypasses Pelecard. To test the real gateway you need Pelecard credentials, and - because Pelecard's iframe is HTTPS and cannot reach localhost - a public HTTPS tunnel to the backend.

Steps

- Install ngrok and expose the backend: `ngrok http 8000` -> gives an https URL.
- Set the values below in `backend/.env` and restart the backend.
- Make sure the club you book has `u_name` matching the credential set you filled (e.g. `matnasim`).
- Book a court with credit card -> Pelecard iframe -> test card (needs ID + CVV) -> redirects -> order confirmed.

```
DEV_MODE=false
PELECARD_GATEWAY_URL=https://gateway.pelecard.biz/Iframe
PELECARD_MATNASIM_TERM=<terminal>
PELECARD_MATNASIM_PASSWORD=<password>
APP_BASE_URL=https://<your-ngrok>.ngrok-free.app # tunnel to the BACKEND :8000
FRONTEND_BASE_URL=http://localhost:3000
```

APP_BASE_URL - where Pelecard redirects after payment:

Mode	APP_BASE_URL
Dev bypass (DEV_MODE=true)	http://localhost:8000 (unused)
Real Pelecard test (DEV_MODE=false)	https://<your-ngrok>.ngrok-free.app (tunnel to :8000)

ngrok's free tier gives a new subdomain each restart - update `APP_BASE_URL` and restart the backend each session.
Live Pelecard terminals charge real money; prefer the PeleTest sandbox for testing.

9. E-mail (SMTP) - testing with Gmail

The site sends transactional e-mail (booking confirmations, cancellations, refund requests, add-to-group, password reset, contact form). For testing on the laptop, send directly through **Gmail** (`servicebaco@gmail.com`). While the credentials are blank, e-mail simply no-ops and the rest of the app keeps working.

One-time Gmail setup

- Sign in to the Google account (`servicebaco@gmail.com`) and turn on **2-Step Verification** (`myaccount.google.com/security`).
- Create an **App Password** at `myaccount.google.com/apppasswords` (pick 'Mail' / 'Other').
Google shows a **16-character** password - copy it (the spaces don't matter).
- Gmail no longer accepts the normal account password over SMTP, so the App Password is required.

Put it in `backend/.env` and restart the backend

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=servicebaco@gmail.com
SMTP_PASSWORD=<the 16-char app password>
EMAIL_FROM=servicebaco@gmail.com
```

Verify it actually sends

```
cd backend
python test_email.py you@example.com
```

Look for **SUCCESS**. If it fails at **login**, the App Password is wrong or 2-Step Verification isn't on. If it fails to **connect**, the network is blocking outbound SMTP (port 587) - common on corporate networks, rare

on a home network (that is why it failed on the work laptop).

Production note: the live site is intended to send via **Mailgun** (smtp.mailgun.org:587, user postmaster@<domain> + the domain's SMTP password) - the provider the legacy system used. The code auto-adapts (STARTTLS only when the server offers it), so switching provider is just .env, no code change. backend/.env.example lists every option.

10. Quick gotchas

- Two servers must run at once: backend :8000 and frontend :3000.
- backend/.env is per-machine - recreate it on the laptop; never commit it.
- Promotion images (frontend/public/promotions/...) come with the clone.
- If the schema is older than the code, run backend/migrations/001_modernization_schema.sql once.
- E-mail: set the Gmail SMTP_* values in .env (section 9); blank = e-mail silently no-ops.
- backend/.env.example is a filled-in template - copy it to .env and edit.